```
    expect(page).to have_selector(
      "tbody:nth-child(2) .name", text: "Take Notes")
    visit(project_path(project))
    within("#task_2") do
      expect(page).to have_selector(".name", text: "Take Notes")
    end
  end
end
```

The failure point is at the first within("#task_3") call, which does not find an element with the DOM ID task_3.

The problem here is timing. The test performs the visit(project_path(project)) then moves immediately to search for within("#task_3"). In the meantime, the code loads the page and makes an Ajax callback to retrieve the project JSON, and the task_3 selector doesn't exist until that call completes. But the test checks and fails well before that can happen.

There are a few ways to handle this issue. For example, you might explicitly put a delay in the test. However, it's easier to use Capybara's built-in ability to wait. The Capybara method find will wait for an element to appear on the page for a default duration of two seconds. This is perfect since it means you can make the test stand still long enough to make sure the Ajax call catches up.

The resulting test looks like this:

js_jasmine/03/spec/system/add_task_spec.rb

```
require "rails_helper"

RSpec.describe "adding a new task" do
  let!(:project) { create(:project, name: "Project Bluebook") }
  let!(:task_1) { create(
    :task, project: project, title: "Search Sky", size: 1, project_order: 1) }
  let!(:task_2) { create(
    :task, project: project, title: "Use Telescope", size: 1,
           project_order: 2) }
  let!(:task_3) { create(
    :task, project: project, title: "Take Notes", size: 1,
           project_order: 3) }

  it "can re-order a task", :js do
    visit(project_path(project))
    find("#task_3")
    within("#task_3") do
      click_on("Up")
    end
    expect(page).to have_selector(
      "tbody:nth-child(2) .name", text: "Take Notes")
```